■ SNAKE GAME

Project Explanation | Python + Turtle Graphics

■ Overview

This is a classic **Snake game** built in Python using the built-in **turtle** graphics library. The player controls a snake that moves around a 600x600 black screen, eats food to grow longer and increase the score, and the game ends if the snake hits the wall or collides with its own tail.

The project is split into **four files**, each handling one clear responsibility — a clean example of Object-Oriented Programming in a small game.

File	Responsibility
main.py	Game loop, controls, collision detection
snake.py	Snake class: movement, growth, direction
food.py	Food class: random spawning
scoreboard.py	Scoreboard class: score display, game over text

■ main.py — The Game Loop

This is the entry point that ties everything together.

- Sets up the screen (600×600, black background, title "SNAKE GAME").
- **screen.tracer(0)** turns off automatic screen updates so movement is controlled manually with **screen.update()** — giving smooth, non-flickery animation.
- Creates one **Snake**, one **Food**, and one **Scoreboard** object.
- **screen.listen()** + **screen.onkey(...)** binds arrow keys to the snake's direction methods.

The Main Loop — runs every 0.1 seconds:

1. Move	snake.move() advances the snake forward one step.
2. Eat	If head is within 15px of food → food respawns, score +1, snake grows.
3. Wall Check	If head passes ±280 on x or y → game over.

4. Tail Check	If head comes within 10px of any body segment → game over.

When the game ends, **scoreboard.game_over()** displays "GAME OVER" and the loop stops.

■ snake.py — The Snake Class

Handles everything about the snake itself.

- **STARTING_POSITIONS** places 3 starting segments 20px apart in a row.
- **create_snake()** builds the initial snake by calling add_segment() for each starting position.
- **add_segment(pos)** creates a white square Turtle, disables its pen, and places it at pos.
- **extend()** adds a new segment on top of the current last segment whenever food is eaten.

■ The Core Trick — move()

This is the heart of the whole game:

```
1. Save the current position of every segment
2. Move the head forward by MOVE_DISTANCE (20px)
3. Each other segment moves to where the
   segment in front of it used to be
```

This creates the classic "follow the leader" snake movement without needing a complicated data structure.

up() / down() / left() / right() change the head's heading — but each checks that the snake isn't currently moving in the exact opposite direction, preventing the snake from instantly reversing into itself.

■ food.py — The Food Class

- Draws a small red circle (shapesize shrinks it to half size).
- speed(0) makes it move/appear instantly with no animation.
- refresh() picks a random (x, y) within the play area (±280) and teleports the food there — called on creation and every time it's eaten.

■ scoreboard.py — The Scoreboard Class

- Positions itself at top-center of the screen (0, 270) and writes the starting score.
- hideturtle() is called after writing so only the text shows, not a turtle icon.
- increase_score() clears old text and rewrites the score every time food is eaten.
- game_over() moves to the center of the screen and writes "GAME OVER" when the game ends.

■ Key Concepts Used

Object-Oriented Programming	Each game element (snake, food, scoreboard) is its own class with its own responsibilities.
Turtle Graphics	Using Turtle objects as visual game pieces, controlled with .goto(), .forward(), .setheading().
Manual Screen Refresh	tracer(0) + screen.update() gives smooth, non-flickering animation instead of turtle's default slow auto-refresh.
Collision Detection via Distance	Collisions are detected by checking if the distance between two turtles is below a threshold — no complex hitboxes needed.
State Tracking Without Extra Structures	The snake's follow-the-leader movement works just by remembering each segment's previous position for one frame.

■ STATUS: FULLY FUNCTIONAL — snake moves, eats, grows, scores, and ends correctly on collision.

■ Possible Future Improvements

- Add a persistent high-score file so the best score is remembered between runs.
- Add sound effects for eating food and game over.
- Add a "press any key to restart" option instead of closing on click.
- Extract magic numbers (15, 280, 10) into named constants for readability.